Task 3 – Choose Variables and Name the Constants

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 3 | Bloom's level: K3 – Apply

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task03)

1. The named constants (single place for every rate)

constants.py

```
# Rate card: change a rate HERE and nowhere else
BASE_RATE = 30.00        # Rs per litre at 3.5 % fat and 8.5 % SNF
FAT_STEP_VALUE = 0.40    # Rs per 0.1 % fat
SNF_STEP_VALUE = 0.30    # Rs per 0.1 % SNF
STD_FAT = 3.5
STD_SNF = 8.5
STEP = 0.1
MIN_FAT = 2.0
MAX_FAT = 7.0
PAISA = 0.01

def rate_per_litre(fat, snf):
    return round(BASE_RATE + round((fat - STD_FAT) / STEP) * FAT_STEP_VALUE
                 + round((snf - STD_SNF) / STEP) * SNF_STEP_VALUE, 2)
```

A rate revision now means editing **one line**; every function reads the name, so a revision can never be applied "partly" as it was in the volunteer's program.

2. Naming rules and conventions

- **Rules (enforced by Python):** a name may contain letters, digits and underscore; it cannot start with a digit (`2value = 5` is a SyntaxError); it cannot be a keyword (`if, for, class`); names are case-sensitive, so `fat_pct` and `Fat_pct` are two different variables.
- **Conventions (PEP 8):** variables and functions in lower_case_with_underscores (`fat_steps`, `rate_per_litre`); constants in UPPER_CASE (BASE_RATE); names must say what the value means (`fat_pct`, not `x`); single letters only for tiny loop counters.

3. Why Python has no true constants

Python has no `const` keyword. Any name can be assigned again, as the session below shows (BASE_RATE became 99 without any error). The language relies on the UPPER_CASE *convention* and on programmer discipline, code review and tests; for a stricter guarantee one can place constants in a separate module (Task 7) and only read them.

4. Dynamic typing, swapping and a type error – interactive session

Python 3.12 interactive mode

```
>>> x = 10
>>> type(x)
<class 'int'>
>>> x = "ten"
>>> type(x)
<class 'str'>
>>> x = 8.5
>>> type(x), x
(<class 'float'>, 8.5)
>>> a, b = 3.5, 8.5
>>> a, b = b, a
>>> a, b
(8.5, 3.5)
>>> "10" + 5
Traceback (most recent call last):
  File "<input>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
>>> BASE_RATE = 30.00
>>> BASE_RATE = 99
>>> BASE_RATE
99
```

```
>>> 2value = 5
  File "<input>", line 1
    2value = 5
    ^
SyntaxError: invalid decimal literal
>>> fat_pct = 4.2; Fat_pct = 3.1
>>> fat_pct, Fat_pct
(4.2, 3.1)
```

- **Dynamic typing:** the *name* x has no type; the *value* does. x was an int, then a str, then a float with no declaration. The risk is that `"10" + 5` fails only when that line runs (TypeError), which is why input must be converted and tested.
- **Multiple assignment and swap:** `a, b = b, a` builds the pair (b, a) first and then assigns, so no temporary variable is needed; a and b changed from (3.5, 8.5) to (8.5, 3.5).

Alternative Concept and Complexity Analysis

Alternative 1 – magic numbers: `30.0 + round((fat-3.5)/0.1)*0.40 + …` typed inside every expression. **Alternative 2 – dictionary:** `RATE_CARD = {"base": 30.0, "fat": 0.40, "snf": 0.30}` read as `RATE_CARD["fat"]`.

Measured (compare_styles.py)

```
magic numbers       734 ns per call
dictionary         1029 ns per call
named constants     752 ns per call
0.40 typed in 1 place per function; a program with k = 3 copies of the formula needs 3 edits
```

	Magic numbers	Dictionary RATE_CARD	Named constants
Places to edit for one revision	k (every copy of the formula; k = 3 in the volunteer's program) – easy to miss one	1	1
Time per rate calculation	≈ 734 ns	≈ 1029 ns (extra hash look-up per value)	≈ 752 ns
Memory	none extra	one small dict (≈ 200 bytes)	three names (≈ 24 bytes each)
Readability / risk	unreadable; numbers have no meaning	good; but a misspelt key fails only at run time (KeyError)	good; a misspelt name is caught on the first run (NameError)

All three are O(1) in time and space for one rate calculation, so running time and memory do **not** meaningfully change (the dictionary is about 40 % slower per call, only 0.3 µs). What changes is the **maintenance cost**: k edits and k chances of error versus one. The named constants are chosen: as fast as magic numbers and as maintainable as the dictionary.

Evidence

One well-named set of constants (constants.py) and an explanation of naming, constants by convention and dynamic typing, backed by an interpreter session. Source: Task03/constants.py, compare_styles.py.